BSE3214 – Cloud Computing and Big Data

WEEK 02

Dynamic Interactions and Computing Architectures

Lecture Material

Part 2 – 15 Hours

Instructor
Odongo Steven Eyobu, Ph.D

Learning Objectives

- Understand scope and control across SaaS, PaaS, and IaaS service models
- Analyze interaction dynamics and user responsibilities for each model
- Map software stack control and shared responsibility boundaries
- Evaluate benefits, challenges, and suitability by service model
- Formulate recommendations and selection criteria for cloud adoption
- Interpret IaaS operational view from provisioning to optimization
- Develop a comparative perspective to guide architecture decisions

Recap – Cloud Service & Deployment Models

- **Service Models:** SaaS (Applications), PaaS (Platform), IaaS (Infrastructure)
- **Deployment Models:** Public, Private, Hybrid, Community
- **Key Emphasis:** User vs. Provider scope of control and shared responsibility

Service, deployment, scope, and control drive interactions and risk profiles.

What is Scope & Control?

Scope

The specific layers of the software stack that you can directly configure and manage.

Control

The degree of authority you possess over system settings, performance tuning, and security configurations.

Trade-offs

More control inherently requires managing greater complexity, increased operational effort, and higher risk responsibility.

Objective

Ideally balance convenience, customization needs, compliance requirements, and cost efficiency.

Layered Cloud Stack Control (Matrix)

Cloud Stack Layer	SaaS Control	PaaS Control	IaaS Control
Application (Data & Config)	Provider	Customer	Customer
Runtime Environment	Provider	Provider	Customer
Middleware	Provider	Provider	Customer
Operating System	Provider	Provider	Customer
Virtualization	Provider	Provider	Provider
Physical Infrastructure	Provider	Provider	Provider

Discussion: Impact on Scope & Responsibility

Higher user control (IaaS) increases flexibility but demands greater security management and operational effort. Lower user control (SaaS) reduces complexity but limits customization. Security is always a shared responsibility regardless of the model.

SaaS Overview

- Complete applications delivered over the network via browsers, mobile apps, or APIs
- Provider owns and operates the entire software stack while customer consumes features
- Multi-tenant architecture by default, enabling shared resources and cost efficiency
- Standardized feature sets with automated, transparent, and frequent updates

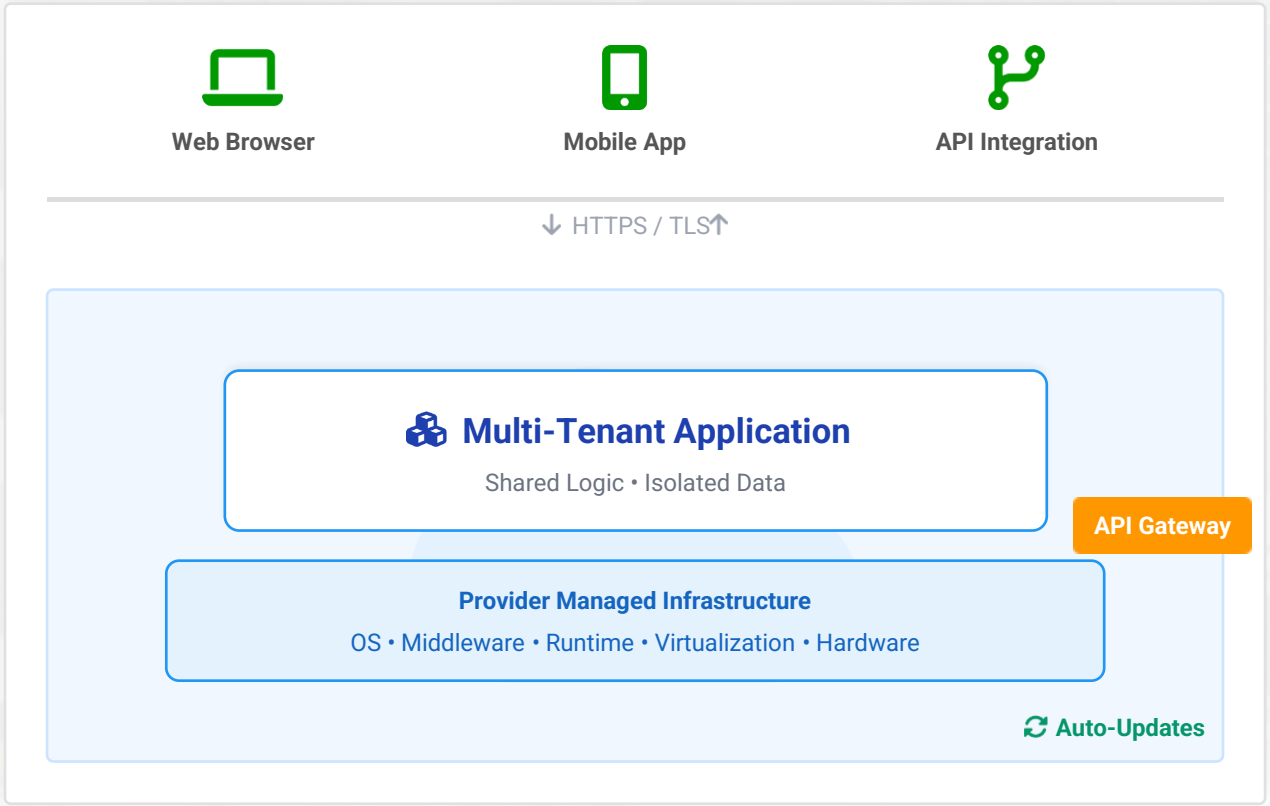
SaaS Software Stack Control

Stack Layer	Customer Control	Provider Control
Data & User Access	Full Control	None
Application Logic	None	Full Control
Runtime Environment	None	Full Control
Middleware	None	Full Control
Operating System	None	Full Control
Infrastructure (Servers, Storage, Net)	None	Full Control

Key Takeaway

In SaaS, the consumer has minimal control over the underlying infrastructure and application platform. Responsibility is primarily focused on managing user access, data governance, and application-level configuration settings.

SaaS Interaction Model



Multi-Tenant Access

Users access a single shared application instance. Logical isolation ensures data privacy while resources are pooled for efficiency.

Transparent Maintenance

Updates, patches, and feature rollouts happen centrally without user intervention. All users are always on the latest version.

Integration & Federation

Interaction extends beyond UI via REST APIs. Identity federation (SSO) allows seamless access control across different SaaS tools.

SaaS Key Benefits

Rapid Provisioning & Deployment

Applications are ready to use instantly. No lengthy installation or configuration processes required for end-users.

Zero Maintenance Overhead

Users never worry about patching, upgrades, or hardware refreshes. The provider manages all backend complexity.

Cost Efficiency (Pay-Per-Use)

Subscription models shift expenses from CAPEX to OPEX. Predictable costs based on actual usage or user count.

Automatic Scalability

Resources expand seamlessly as user demand grows without manual intervention or service interruption.

Global Accessibility & High Availability

Access applications securely from any device, anywhere via internet. Providers offer robust SLAs ensuring uptime and reliability across geographic regions.

SaaS Issues and Concerns

Vendor Lock-in & Migration Complexity

Proprietary formats and APIs can make it difficult to switch providers. Extracting data (egress) often involves technical challenges and costs.

Limited Customization & Integration

One-size-fits-all approach limits deep customization. Integrating with legacy on-premise systems may require complex middleware.

Data Security, Privacy & Compliance

Sensitive data resides outside the organization's firewall. Compliance with regulations (GDPR, HIPAA) depends heavily on the provider's controls.

Dependency on Provider SLAs

Business continuity relies entirely on the provider's uptime. Outages, performance degradation, or service discontinuation are risks beyond user control.

SaaS Suitability & Recommendations

Best Suited For

Standardized Business Processes

Ideal for common functions like email (Gmail, O365), CRM (Salesforce), HR, and collaboration tools where customization needs are minimal.

Minimal IT Overhead

Perfect for organizations that want to focus on core business rather than managing software updates, patching, and infrastructure.

Distributed Workforce

Supports remote and mobile teams needing secure, anywhere access via web browsers or mobile apps without VPN complexity.

Key Recommendations

Review Compliance & SLAs

Thoroughly vet provider certifications (ISO 27001, SOC 2) and understand uptime guarantees, support response times, and penalties.

Plan Integration & Portability

Evaluate API capabilities for connecting with existing systems and define an exit strategy to avoid vendor lock-in risks.

Security Best Practices

Implement Single Sign-On (SSO) and Multi-Factor Authentication (MFA). Ensure sensitive data is encrypted both in transit and at rest.

PaaS Introduction

- **Developer-centric platforms** providing comprehensive OS, runtime, middleware, development tools, and APIs ready for use
- **Automates infrastructure management** including scaling, load balancing, patching, and underlying maintenance tasks
- **Supports cloud-native patterns** enabling modern architectures like 12-factor apps, microservices, and container orchestration
- **Accelerates delivery** by removing infrastructure friction, allowing teams to focus purely on application logic and innovation

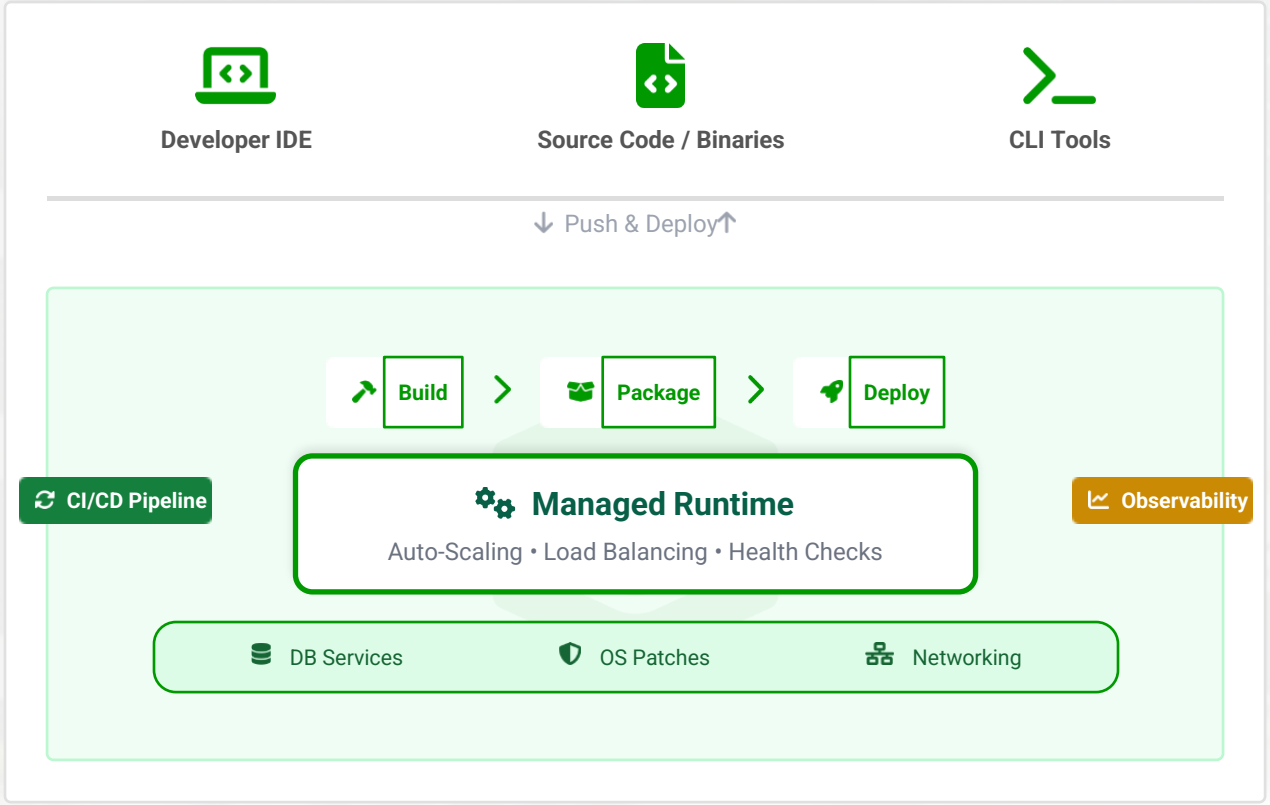
PaaS Software Stack Control

Stack Layer	Customer Control	Provider Control
Application (Code & Data)	Full Control	None
Runtime Environment	Partial Control	Mostly Controls
Middleware	None	Full Control
Operating System	None	Full Control
Virtualization	None	Full Control
Infrastructure (Servers, Storage, Net)	None	Full Control

Key Takeaway

In PaaS, responsibility is shared. Customers focus on application code, data, and runtime parameters (versions/configurations), while the platform provider abstracts and manages the underlying middleware, OS, and infrastructure layers.

PaaS Interaction Dynamics



Developer-Centric Workflow

Developers push code or binaries directly. The platform handles building, containerizing, and deploying the application automatically.

Integrated Tooling

Seamless integration with IDEs, APIs, CI/CD pipelines, and observability hooks creates a streamlined development lifecycle.

Automated Management

The platform automatically manages the underlying runtime, middleware, and OS patching, removing operational burden from developers.

PaaS Benefits

Accelerates Development & Deployment

Streamlines the coding lifecycle with pre-configured environments, allowing developers to focus on application logic rather than infrastructure setup.

Reduced Infrastructure & Ops Overhead

Eliminates the need for manual server management. Middleware, patching, and runtime environments are fully managed by the provider.

On-Demand Scalability

Supports advanced deployment strategies like blue/green deployments and canary releases, automatically adjusting resources to match traffic demands.

Standardized Pipelines & Collaboration

Enables consistent CI/CD workflows across development teams, fostering better collaboration and reducing environment inconsistencies.

PaaS Issues and Concerns

Vendor & Platform Lock-in

Dependency on proprietary APIs, buildpacks, and platform-specific services can make migrating applications to another provider costly and complex.

Limited OS & Middleware Tuning

Lack of access to underlying operating system and kernel-level configurations restricts performance tuning for specialized workloads.

Portability Challenges

Moving applications across different PaaS providers is difficult due to varying runtime environments, supported languages, and service integrations.

Debugging Complexity

Abstraction layers hide underlying infrastructure details, making deep troubleshooting and root cause analysis more challenging compared to IaaS.

PaaS Suitability & Recommendations

Best Suited For

Agile Teams & Startups

Ideal for teams needing rapid prototyping and continuous delivery cycles without managing underlying infrastructure complexity.

Cloud-Native Applications

Perfect for developing scalable, resilient applications using microservices architecture and event-driven patterns.

Continuous Integration/Deployment

Streamlines DevOps processes with built-in CI/CD pipelines, automated testing, and seamless deployment workflows.

Key Recommendations

Prefer Open Standards

Choose platforms supporting open standards and containers (Docker, Kubernetes) to ensure portability and avoid vendor lock-in.

Validate Ecosystem Integration

Ensure the platform supports required languages, runtimes, and integrates smoothly with existing tools and services.

Define SLOs & Observability

Establish clear Service Level Objectives (SLOs) and instrument applications for comprehensive monitoring and observability.

IaaS Overview

- **Virtualized Resources:** Compute, storage, and networking delivered as on-demand services
- **User Responsibility:** Full management of Guest OS, middleware, runtime environments, and applications
- **Provider Responsibility:** Management of physical data centers, hardware, networking infrastructure, and hypervisor layer
- **Key Characteristic:** Maximum flexibility and control over the computing environment, similar to traditional on-premise IT

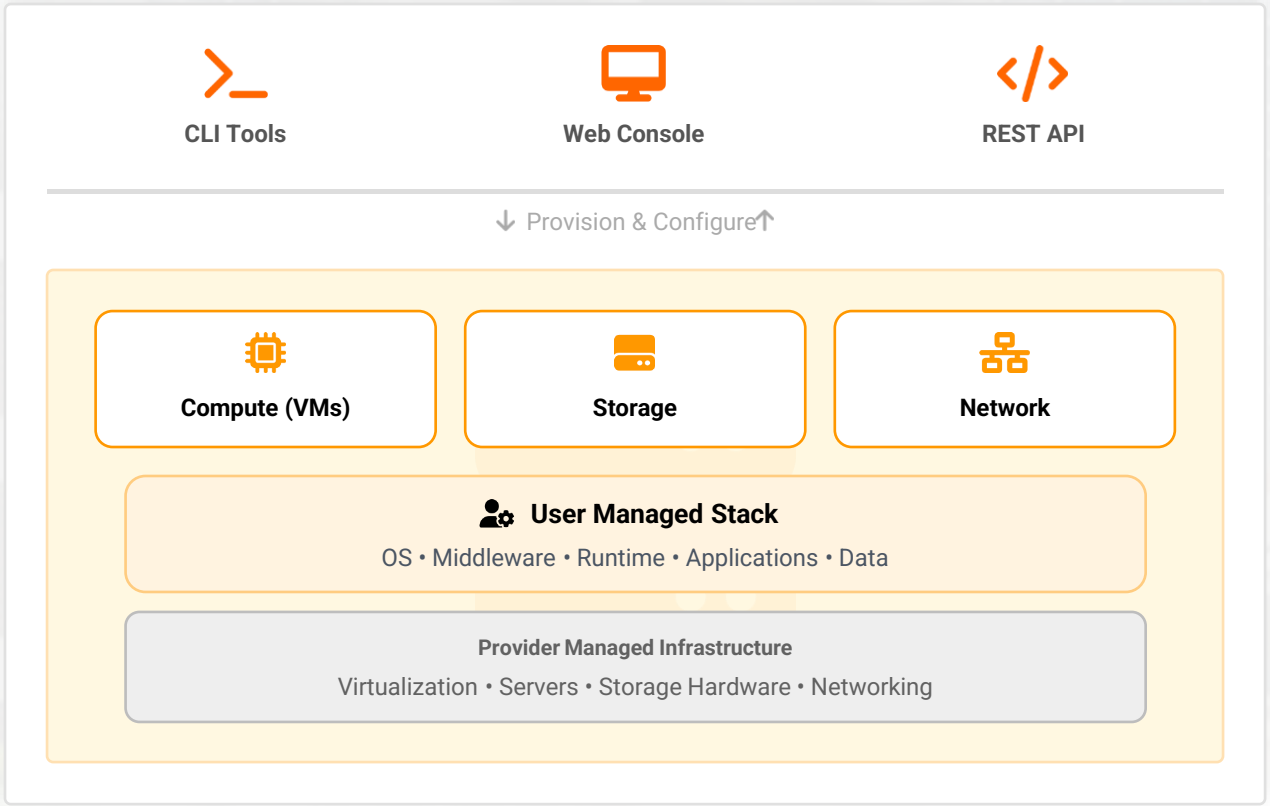
IaaS Software Stack Control

Stack Layer	Customer Control	Provider Control
Application Logic	Full Control	None
Runtime Environment	Full Control	None
Middleware	Full Control	None
Operating System	Full Control	None
Virtualization	None	Full Control
Infrastructure (Servers, Storage, Net)	None	Full Control

Key Insight: Maximum Flexibility = Maximum Responsibility

In IaaS, the customer gains full control over the software stack from the OS upwards. This provides maximum flexibility for custom configurations but also places the burden of patching, security hardening, and maintenance squarely on the customer.

IaaS Interaction Dynamics



Provisioning & Management

Users provision raw resources via APIs, CLI, or Console. Full control to install OS, configure networks (VPCs, Subnets), and deploy any software stack.

Security Configuration

User is responsible for defining Security Groups, NACLs, firewalls, and routing tables. Granular control over network traffic and access.

Automation & IaC

Operations can be fully automated using templates (CloudFormation, Terraform) and images (AMIs) for reproducible infrastructure deployment.

IaaS Operational View

1

Provision Resources

Instantiate Virtual Machines (VMs), allocate storage volumes, and define network boundaries.

2

Configure OS

Install patches, harden the Operating System, and configure user accounts/SSH keys.

3

Deploy Apps

Install middleware, runtime environments, and deploy application code/binaries.

4

Secure & Compliance

Configure Identity & Access Management (IAM), firewalls, security groups, and encryption keys.

5

Monitor & Backup

Set up logging, performance metrics, disaster recovery snapshots, and automated backups.

6

Scale / Decommission

Optimize resource usage based on demand, scale horizontally/vertically, or terminate unused instances.

Recommended Tooling: Leverage Infrastructure as Code (IaC) tools like **Terraform** for provisioning, Configuration Management tools like **Ansible** for OS setup, and **CloudWatch/Prometheus** for monitoring.

IaaS Benefits

Maximum Flexibility & Customization

Offers complete control over the computing environment, ideal for complex workloads that require specific configurations unavailable in PaaS or SaaS.

Legacy Application Support

Suitable for migrating legacy and bespoke enterprise applications without rewriting code ("lift and shift"), maintaining existing architectures.

Granular Security & Compliance

Enables deep security customization at the OS and network level, allowing organizations to meet strict regulatory compliance requirements.

Elastic Capacity & Right-Sizing

Dynamically scale resources up or down based on actual workload needs, optimizing performance while controlling infrastructure costs.

IaaS Issues and Concerns

Requires Specialized Expertise

Demands deep technical knowledge in OS administration, networking, security, and application management. Staff must be skilled in cloud architecture.

Security Misconfiguration Risks

Users are fully responsible for securing the OS and apps. Misconfigured firewalls or unpatched systems can lead to severe security vulnerabilities.

Operational Complexity & Toil

Higher operational overhead compared to PaaS/SaaS. Requires significant effort for maintenance tasks like patching, backups, and monitoring.

Cost Management Challenges

Without proper governance, resource tagging, and monitoring, costs can spiral out of control due to unused resources or inefficient scaling.

IaaS Recommendations & Best Practices

Use Infrastructure as Code (IaC)

Define and provision infrastructure using code (e.g., Terraform, CloudFormation) to ensure consistency, version control, and rapid replication of environments.

Implement Strict IAM & MFA

Enforce least privilege access principles. Require Multi-Factor Authentication (MFA) for all root and administrative user accounts to prevent unauthorized access.

Enforce Patching, Backups & DR

Automate OS patching schedules to close vulnerabilities. Maintain regular backups and test Disaster Recovery (DR) procedures periodically.

Monitor Usage & Costs

Set up budget alerts to detect cost anomalies early. Use tagging strategies to track resource consumption by department, project, or environment.

Follow Security Benchmarks

Adhere to cloud provider security best practices and industry standards like CIS Benchmarks to harden configurations against common threats.

SaaS vs PaaS vs IaaS Comparison Summary

Characteristic	SaaS	PaaS	IaaS
Control Level	**Minimal** (Application usage & Data configuration only)	**Medium** (Applications, Data & Runtime settings)	**Full** (OS, Middleware, Runtime, Data & Applications)
User Responsibility	Data governance, User access, Client-side config	Application code, Data, Runtime environment params	OS patching, Middleware, Runtimes, Apps, Data, Security
Provider Responsibility	Entire stack: App, Data, Runtime, Middleware, OS, Infra	Runtime, Middleware, OS, Virtualization, Servers, Storage	Virtualization layer, Physical Servers, Networking, Storage
Customization Level	Low (Configuration only)	Moderate (App logic & extensions)	High (Full OS & Software stack)
Ideal User	End Users / Business Units	Software Developers	System Admins / Network Architects
Complexity	Lowest	Medium	Highest

Real-World Use Cases & Suitability

SaaS Examples

Email & Collaboration
Google Workspace (Gmail), Microsoft 365, Slack, Zoom

CRM & Sales
Salesforce, HubSpot, Zoho CRM

Storage & Productivity
Dropbox, Box, Adobe Creative Cloud

PaaS Examples

Web App Platforms
Heroku, Google App Engine, Azure App Service

Container Management
Google Kubernetes Engine (GKE), Azure Kubernetes Service (AKS)

Backend Services
Firebase, AWS Lambda (Serverless), Azure SQL Database

IaaS Examples

Legacy Migration
Lift-and-shift of existing enterprise applications to cloud VMs

High Performance Computing
Scientific simulations, financial modeling requiring specialized hardware

Custom Networks
Complex VPC architectures with custom routing, firewalls, and VPNs

Deployment Model Overview

Public Cloud

Infrastructure is provisioned for open use by the general public. Owned, managed, and operated by a third-party provider (e.g., AWS, Azure).

Key: Rapid Scale & Pay-as-you-go

Private Cloud

Provisioned for exclusive use by a single organization. Can be on-premises or hosted remotely, offering maximum control and customization.

Key: Dedicated Control & Compliance

Hybrid Cloud

Composition of two or more distinct infrastructures (private, community, or public) that remain unique entities but are bound by standardized technology.

Key: Balance of Scale & Sovereignty

Community Cloud

Provisioned for exclusive use by a specific community of consumers from organizations that have shared concerns (e.g., mission, security requirements).

Key: Shared Costs & Common Goals

Critical Consideration: Always evaluate **Data Residency** requirements. Sovereign cloud options may be necessary for government or highly regulated industries to ensure data remains within national borders.

Security Considerations

Multi-Tenancy Risks

Isolation failure risks where unauthorized access across tenants could occur. "Noisy neighbor" performance impacts primarily in SaaS and public PaaS models.

Data Protection & Encryption

Mandatory encryption for data at rest (storage) and in transit (TLS). Customer-managed keys (KMS) provide sovereignty over data access.

Identity & Access Management (IAM)

Enforce Least Privilege access. Mandate Multi-Factor Authentication (MFA). Use Single Sign-On (SSO) and federation to centralize identity control.

Resilience & Incident Response

Automated backups and Disaster Recovery (DR) testing. Centralized logging (CloudTrail/CloudWatch) for auditing and rapid incident response.

Shared Responsibility Model: Security is always a joint effort. The provider secures the infrastructure (of the cloud), while you secure your data and configuration (in the cloud).

Discussion Questions

How do you choose between SaaS, PaaS, and IaaS for a new workload?

What are your organization's biggest concerns in cloud adoption?

How do the interaction dynamics affect security policies and controls?

Can vendor lock-in be mitigated effectively? If so, how?

Summary & Next Steps

Key Takeaways

- **Scope & Control:** Understand the varying degrees of control and shared responsibility across SaaS, PaaS, and IaaS models.
- **Business Alignment:** Match the benefits and trade-offs of each service model to your specific business needs, risk tolerance, and skill gaps.
- **Governance Planning:** Establish robust governance for operations, security, and cost management from day one.
- **Future Proofing:** Prepare for hybrid and multi-cloud architectures by prioritizing portability and open standards.

Interesting previews

Cloud Architectures

Deeper dive into cloud-native architectural patterns and microservices.

Orchestration & Automation

Exploring Kubernetes, Terraform, and CI/CD pipelines for automated deployment.

Lab Preparation

Ensure your AWS/Azure student accounts are active for upcoming hands-on exercises.